Smart Lab Portal — Status Report

BSc thesis (GIU) · 14 October 2026, evening · branch `offline-router-door-photos`, HEAD `802638b` + uncommitted Command Center fix

> **Labels used: LIVE-VERIFIED** = measured on the real hardware/live system · **TESTED OFFLINE** = automated tests or simulation/recordings, no hardware · **UNVERIFIED** = not yet measured. No hardware result in this report is estimated; numbers come from logs and measurements.

1. Summary

- **UR5e digital twin** is LIVE-VERIFIED (read-only RTDE, 48 Hz, joint mapping checked joint by joint, stale detection 2.0 s).
- **FANUC ER-4iA digital twin** is LIVE-VERIFIED (read-only HTTP GET of `/MD/CURPOS.DG`, ~104 ms per read, ~3 Hz; the twin followed the pendant). The J2/J3 coupling still needs a pose test.
- **Camera 1** ran live at 7 analysis passes/s on the GPU. The 14 Oct tracking/identity overhaul is deployed but not yet confirmed live (the lab closed).
- **Camera 2 (MC200 1080p)**: software ready and tested with two simulated cameras; the device arrives tomorrow.
- **Performance**: a full simulated load test now exists. One real bottleneck was found and fixed (Command Center reloaded all its data on every live message); no backend or database bottleneck was found.
- **Safety**: robots stay read-only, the door is opened only by the Master ESP32, production data untouched, nothing pushed.

2. Work completed (12–14 October)

Robots

- UR5e collector (`tools/robot_monitor`, Python 3.12 + ur_rtde, receive only): connection check, validity check (6 finite joints), controller-timestamp de-duplication, movement events, home calibration, stale detection.
- FANUC source: only an HTTP GET of the current-position page; parser checked against the real controller page (R-30iB Mate Plus, HandlingTool V9.30); world-frame TCP only; motion detected on the robot's own joints; the twin-only J3+J2 coupling applied only for drawing. A test proves the source class has no write path (no POST/PUT, sockets, KAREL, register writes).
- FANUC twin shows the Schunk EGP 40 electric gripper (drawn approximately). `start_fanuc_monitor.bat` and `launch.bat` start the monitor when `FANUC_IP` is set in the git-ignored `robot.env`.

Security camera and identity

- GPU (DirectML, GTX 1650): YOLO26s detector, one shared detector session and a single GPU lock — this fixed the `nvwgf2umx.dll` driver crash caused by a second DirectML session.
- Analysis loop: error handling, live fps, Windows QuickEdit disabled (clicking the console no longer freezes the camera), per-port guard against double processes.
- Tracking: thin duplicate boxes dropped; still people keep their track 6 s; appearance check after a gap. On real footage, broken tracks fell from 12 to 4.
- Identity rules: door entry, face, look, place, fresh-only elimination, home desk (shown only as a guess "? (desk)", never assigned on desk alone, yields to a newer door entry). Identity decisions logged per

camera.

- Two cameras: per-camera env file, port, window title and logs; all `camera*.env` git-ignored; credentials masked in all output.

Backend, portal and data

- Environment "latest values" query rewritten (newest per lab and metric): query 72 → 2.1 ms, function 92 → 11.6 ms, live `/monitor` 97 → 59 ms (deployed).
- Test isolation: tests are forced onto `smartlab_test` with an advisory lock so two runs cannot deadlock.
- n8n workflows 01/03/05/07 recovered byte-exact and the repo copy aligned with the live one (live workflows not edited).
- Backups: snapshot branch, uncommitted-work patch, database dump before the deploy, n8n recovery folder, lab evidence screenshots.

This session (performance mission)

- New repeatable load test: `tools/perf/load_simlab.py`, `tools/perf/ws_clients.mjs`, guide `docs/perf/LOAD_TEST.md`. Runs on a separate backend copy and database (`smartlab_loadtest_test`, port 8100) — refuses the live portal.
- Fix in `frontend/src/pages/CommandCenter.tsx` (uncommitted, not deployed): see section 4.

3. Measurements (all SIMULATED load, isolated stack)

Workload for 120 s: 2 simulated robots (UR5e + FANUC models, ~9.5 telemetry posts/s), 2 camera services on recorded lab video (GPU), an environment node every 2 s, 3 WebSocket clients, and dashboard polling **faster than the real portal** (worst case). Live camera 1 kept running alongside.

Endpoint (server side)	p50 ms	p95 ms	Note
/health	13–26	30–92	
/robots, /labs/hazards/active	19–21	46–59	
/robots/:id/events	35	86	
/security/cameras, /system/status	29–43	71–99	
/summary	52	143	
/monitor	72–91	204–251	with no browser open; 0.8–1.0 s p95 while a browser also loaded the stack

- API errors: **0** in every run. WebSocket: 9.7 messages/s and 2.2 KB/s per client (robot_state 7.6/s, capped per robot).
- Two simulated cameras: 6.5–6.7 analysis passes/s each, pass p95 170–282 ms, frame age p95 84–95 ms, GPU ~65%, VRAM ~1.0 GB, system CPU ~69%.
- `/monitor` breakdown: ~22 ms idle (15 queries; the slowest query is 5.9 ms). Under load, measured from a separate process, it took ~51 ms: the extra time is the laptop's CPU being busy (cameras), not slow queries. The backend container used 20–56% of one core — not saturated.

4. Bottleneck found and fixed

> **Command Center (/command) reloaded all 8 of its data endpoints on every live status message** (camera state, environment update, robot status), after only a 400 ms delay; every robot status also re-fetched robot events immediately. With a camera, robots and the environment node reporting, the page reloaded about once per second.

- **Fix:** safety messages (hazard, alert, door) still refresh within 400 ms exactly as before. Status messages now refresh at most every 3 s, and the latest one is always applied (never dropped). Robot events follow the same 3 s limit. No alarm is throttled or discarded; the separate hazard alarm component is unchanged.
- **Result (TESTED OFFLINE, same load):** /command browser requests 13.5 → 9.4 /s; /monitor 0.8 → 0.4 /s; robot events 1.5 → 0.7 /s. Type check and production build pass.
- **Not yet deployed:** it goes live only after your approval of a commit and a `docker compose build frontend` + restart of the frontend container.

Remaining performance notes

- /command shows long main-thread tasks in headless Chrome, but a CPU profile shows 12.4 of 20 s is native rendering: headless Chrome has no GPU, so the 3D twin runs in software WebGL. The page's own JavaScript is small (~0.6 s per 20 s). The twin's cost in a normal browser with a GPU is UNVERIFIED.
- The `/api/health` times measured inside the /command page (300–560 ms) are caused by that blocked main thread; the server answers the same call at p95 < 100 ms.
- Each robot telemetry post writes the robot's row and commits (~10/s for two robots). It is within capacity today; batching it would change how "last seen" works, so it was not changed.

5. Why two different latency numbers ("8×" vs "97 → 59 ms")

They measure different things. **"8×"** is the single environment function on its own (92 → 11.6 ms). **97 → 59 ms** is the whole live `/monitor` request: login check, about 15 other queries, building the tiles and sending the JSON — only one part of it got faster. Both are correct for their scope.

6. Acceptance matrix

Item	Result	Evidence / reason
UR5e live twin (read-only)	**PASS**	LIVE-VERIFIED 14 Oct: 48 Hz, each joint checked, stale in 2.0 s
FANUC live twin (read-only)	**PASS**	LIVE-VERIFIED 14 Oct: twin followed pendant jogging; GET only
FANUC J2/J3 coupling on twin	**NOT TESTED**	needs the pose test in section 7
UR5e + FANUC at the same time	**NOT TESTED**	tested only in simulation (2 sim robots)
Camera 1 live detection on GPU	**PASS**	LIVE-VERIFIED 14 Oct, 7 passes/s
Camera 1 new tracking/identity (14 Oct)	**NOT TESTED**	deployed; tested on recordings only (148 tests pass)
Camera 2 MC200	**BLOCKED**	device arrives tomorrow; two-camera software tested offline
Cross-camera identity hand-over	**FAIL**	not implemented
Full-system load test	**PASS**	TESTED OFFLINE, 0 errors, see section 3
Command Center refresh fix	**PASS**	TESTED OFFLINE; awaiting deploy approval
3D twin smoothness in a real browser	**NOT TESTED**	headless measurement not representative
Door / Master ESP32 flow	**PASS**	validated earlier; unchanged
Robot test suite	**PASS**	32 passed (this session)
Camera test suite	**PASS**	148 passed (this session)
Backend test suite	**PASS**	436 passed at the 14 Oct deploy; backend unchanged since, not rerun
Frontend type check + build	**PASS**	after the fix (this session)
FANUC controller battery (DIAG-009)	**FAIL**	battery to replace — section 8

7. Physical acceptance checklists

For all of these: the portal only reads. Motion is done only by the robot operator with the pendant, under the lab's normal safety rules.

FANUC ER-4iA — J2/J3 test

1. Start the FANUC monitor; confirm the twin shows "FANUC live" with a fresh age.
2. Operator jogs **J2 only** by about +20° in JOINT mode. Check: the console reports J2 moved and **not** J3; on the twin, does the forearm keep its angle to the ground (coupling right) or rotate with the upper arm (coupling wrong)?
3. Jog **J3 only** by about +20°. Only the forearm should move on the twin.

4. Take a photo of the real arm and a screenshot of the twin in the same pose. Record the pendant joint values for both poses.
5. Repeat J1, J4, J5, J6 one at a time (direction and size).

UR5e

1. Cable to the Smart_Lab router; robot IP 192.168.1.102; start the UR5e monitor.
2. Check rate ~48 Hz and joint directions once more after the move to the shared router.
3. Unplug the cable: the twin must show TELEMETRY_STALE within ~2 s, and recover when plugged back.

Both robots together

1. Run both monitors; both twins fresh at the same time on /command.
2. Jog one robot: only its own twin moves. Note the portal latency and CPU with both live plus the camera.

Camera 1 (MC210)

1. Start the camera; check GPU engines and ~7 passes/s in the metrics.
2. Walk in through the door: your name stays on you; sit next to another person: labels do not swap.
3. Sit at someone else's desk: you are not given their name; the desk guess shows only "? (desk)".

Camera 2 (MC200 1080p)

1. Set its IP and stream on the Smart_Lab router; copy `camera2.env.example` to `camera2.env` and fill the source locally (never commit).
2. Start via `launch.bat`; check port 5101, both cameras ~7 passes/s, GPU load.
3. Note: a person moving between the two cameras is not matched yet.

8. DIAG-009 — FANUC controller battery

The pendant shows **DIAG-009**, which you reported as a battery warning on the controller. The exact message text was not captured — take a photo of the alarm line next time.

- Do not ignore it: if the backup battery runs flat, the controller can lose its memory, or the robot can lose its position calibration (mastering) and need re-mastering by a qualified person.
- Before replacing: make a full controller backup (image / all files) to a USB stick.
- Replace the battery the FANUC maintenance manual specifies for this controller and robot, following the manual's procedure. For FANUC robot (encoder) batteries the controller is normally kept **powered on** during replacement. Check this in the manual for your model — UNVERIFIED here.
- This is lab-staff work; the portal and this project never change controller settings.

9. What needs your approval

1. Commit the Command Center fix, the load-test tools and `docs/perf/LOAD_TEST.md`.
2. Rebuild and restart the frontend container so the fix goes live.
3. Pushing to GitHub and the password changes remain postponed, as you asked.

10. Next

- New mission received: faster ESP32-CAM face enrollment and door recognition — starts with profiling the existing flow (no security thresholds lowered).

- At the lab: FANUC J2/J3 test, UR5e on the shared router, both robots together, camera 1 identity check, MC200 setup.